Welcome to the Introduction to NoSQL course! 👋

You are about to dive into the exciting world outside relational databases… 🐙

About the course

- This course focuses on the diverse and widely used family of non-relational database management systems, commonly called **NoSQL database management systems**
- We will learn the core principles of NoSQL DBMSs, how they differ from relational DBMSs, and their main benefits and challenges
- The main technology in the course is **MongoDB**
- We will cover MongoDB data management principles, data modeling, database operations, and database programming in Python
- The prerequisites are the **Data Management and Databases** and **Python Programming** courses

Learning objectives

- After completing this course, you know how to…
  - Explain the main characteristics of NoSQL database management systems
  - Explain the main differences between NoSQL and relational DBMSs
  - Provide examples of the main application areas of NoSQL DBMSs
  - Explain the principles of the data model in the MongoDB DBMS
  - Install the MongoDB DBMS and the MongoDB Compass tool
  - Create a database and perform database operations with the MongoDB DBMS
  - Implement a simple Python application that operates on a MongoDB database

Materials and exercises

- The course material and exercises consist of **four parts**, which should be completed in order:
  - The first part is an **introduction to NoSQL DBMSs** with a multiple-choice Moodle exam
  - The second part is an **introduction to MongoDB DBMS** with a multiple-choice Moodle exam and practical setup exercises
  - The third part covers **MongoDB data modeling and database operations** with practical query exercises
  - The fourth part covers **database programming in Python** with a small programming project
- Coursework is submitted in Moodle, while the materials and exercises are in GitHub (link in Moodle)

Schedule and assessment

- The course schedule is flexible, and there are only **two deadlines**:
  - Passing the Moodle exam of the first part **before Friday 3.4. at 8:00**
  - Finishing everything else **before Sunday 17.5. at 23:59.**
- The course is assessed as **pass or fail**. A passing grade requires passing both Moodle exams and completing the other exercises according to their requirements
- If your submission has issues, the teacher will request changes and you have a chance to resubmit your work

Brief history of NoSQL databases

- As the internet expanded in the early 2000s, major companies such as Google and Amazon faced challenges in maintaining query performance with an ever-growing number of simultaneous users and highly varied data
- Traditional **relational DBMSs** struggled to meet these demands, particularly in terms of scalability, flexibility, and performance
- To address these limitations, a new class of **highly scalable** systems, known as **NoSQL DBMSs**, began to emerge from within these major companies
- One of the core design principles of many NoSQL DBMSs has been **horizontal scaling**, which distributes data storage and computation to multiple machines in the system
- Since the 2010s, **the NoSQL ecosystem has grown rapidly**, with many NoSQL DBMSs establishing themselves among the most widely adopted DBMSs in the world

What are NoSQL DBMSs?

> *"When people use the term "NoSQL database", they typically use it to refer to any non-relational database. Some say the term "NoSQL" stands for "non-SQL" while others say it stands for "not only SQL"."*

- The term *"NoSQL"* commonly refers to the large family of DBMSs that **are not relational**, meaning the data model **is not based on tables, relationships, and join operations**
- Instead, data can be structured in multiple ways depending on the NoSQL DBMS, such as **JSON documents**, **key-value pairs**, and **graphs**
- Different NoSQL DBMSs provide **solutions for many different data requirements**, for example extremely fast key-value access without the overhead of managing complex table relationships
- Next, let's look at three widely used NoSQL DBMSs and their data model approaches: **MongoDB**, **Redis**, and **DynamoDB**

MongoDB — Document database

- **MongoDB** is a **document database** that stores data in **collections** containing flexible, JSON-like **documents**:

```
{
  "_id": "68c3b961da0f719a26014775",
  "title": "Dune",
  "author": "Frank Herbert",
  "published": 1965,
  "genres": ["Science Fiction", "Adventure", "Politics"],
  "copies": 7,
  "ebook": false,
  "location": {
    "section": "Science Fiction",
    "shelf": "S2"
  }
}
```

MongoDB — Document database

- Database operations are performed using the **MongoDB Query Language** (MQL), which has a very JavaScript-like syntax:

```
db.books.find({ "author": "Frank Herbert", "year": { "$gte": 1965 } })
```

- The key benefits of MongoDB are its **flexibility** and **horizontal scalability**
- The JSON-like documents **don't have a fixed schema** (e.g. predefined field names and data types), which makes it easy to evolve the structure
- MongoDB can handle high data volumes and traffic efficiently due to **its ability to scale using sharding and replication**
- These benefits make MongoDB a good choice for content management systems (flexibility requirements due to varying data) and real-time applications (high scalability requirements due to high data volumes)

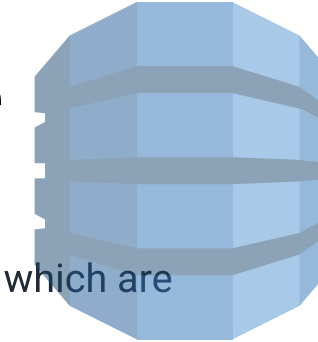
Redis — Key-value database

- **Redis** is a **key-value database** that stores data as **key-value pairs**, where a unique string key maps to a value in a flexible format
- The queries focus on key-based operations, e.g. accessing the value of a specific key:

```
SET teacher:h02680 "h02680;Kalle;Ilves;kalle.ilves@haaga-helia.fi"
GET teacher:h02680
```

- Redis has a simpler data model than MongoDB, but its core operations (e.g., `SET` and `GET`) are **extremely fast due to its in-memory design**
- Data is stored in main memory for fast access, while optionally persisting it to disk for permanent storage
- Due to its fast key-based access, Redis is commonly used as a **secondary caching layer** to store frequently accessed, slow-to-query data, reducing latency and offloading the primary database

DynamoDB — Key-value and document database



- **Amazon DynamoDB** is a **key-value and document database** managed in AWS
- The data is stored in **tables** (different from relational DBMSs) that contain **items**, which are flexible JSON documents identified by a primary key
- Database operations are performed using the **DynamoDB API**:

```
{
  "TableName": "Orders",
  "KeyConditionExpression": "CustomerId = :id",
  "ExpressionAttributeValues": {
    ":id": { "S": "cus_7mbrsagJ7" }
  },
  "ProjectionExpression": "OrderId, Items, ShippingAddress"
}
```

- DynamoDB's queries are primarily designed around **fast key-based access**, in contrast to MongoDB, which supports flexible and complex queries across many fields

Trade-offs in NoSQL DBMSs

- The benefits of NoSQL DBMSs come with drawbacks, and **trade-offs need to be made to gain certain benefits**
- **Horizontal scalability** offers performance and fault tolerance, but often trades away **strong consistency**. Written data is only *"eventually"* propagated across all machines in the system and until then, users may read **stale, outdated data**
- **Schema flexibility** makes it easy to evolve the data model, but often leads to **low data quality**, since no strict rules are enforced on the structure of the data (though enforcement is possible in some NoSQL DBMSs, such as MongoDB)
- In many scenarios, however, these trade-offs are reasonable and can still meet the data requirements

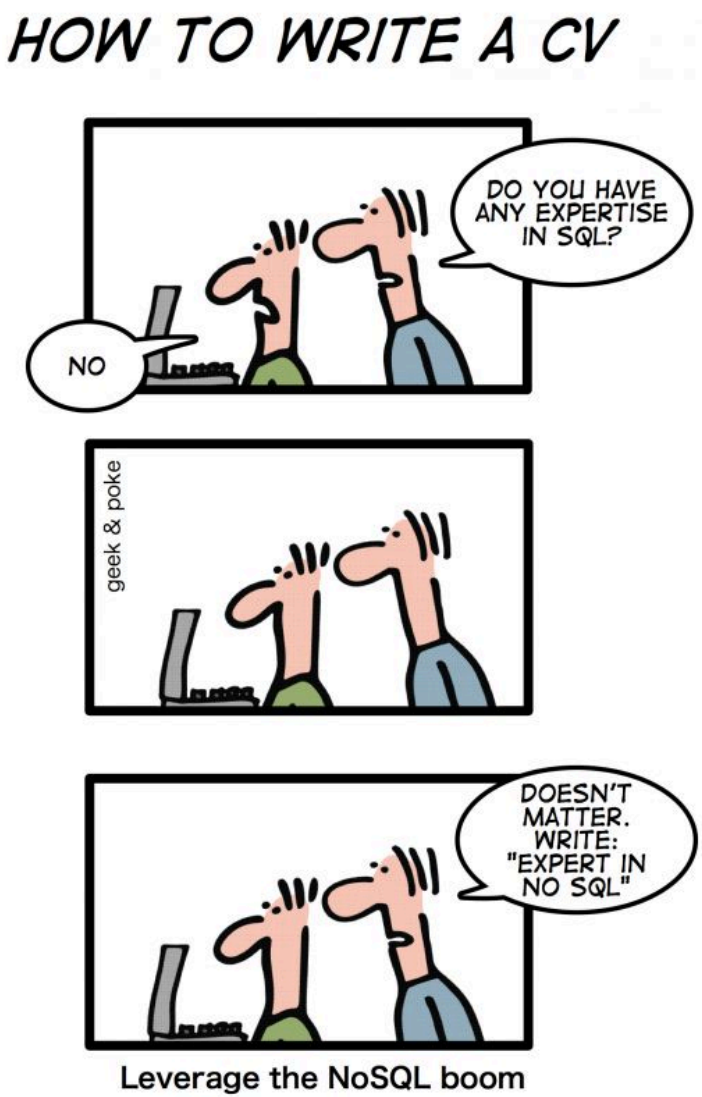
Why should we study NoSQL DBMSs?

> *"It is tempting, if the only tool you have is a hammer, to treat everything as if it were a nail."*

- As we saw from these examples alone, the **NoSQL DBMS family offers a wide range of different kinds of tools** suitable for different kinds of use-cases
- For example, Redis often offers much better performance for key-value operations than relational RDBMSs
- MongoDB, on the other hand, offers database schema flexibility and built-in scalability
- Often, a system's architecture **consists of more than one DBMS**, for example MongoDB as the primary database and Redis as the cache layer
- Studying NoSQL DBMSs offers us more tools to solve different kinds of problems

The popularity of NoSQL DBMSs

- The "State of Database Survey" annually studies the usage of different database technologies. Here are a few highlights from the year 2023:
  - **MongoDB** and **Redis** NoSQL DBMSs are in the **top four most widely used DBMSs** right after PostgreSQL and MySQL
  - 74.4% of respondents have used MongoDB and 73.3% have used Redis
- This indicates that **NoSQL DBMSs are widely acknowledged and used in the industry** despite not being as popular as certain relational DBMSs
- In web development, different kinds of **tech stacks containing MongoDB are widely adopted**
- MERN (MongoDB, Express, React, Node) and MEAN (MongoDB, Express, Angular, Node) are acronyms for such common tech stacks



Closing words & QA

- Thanks for joining the session and again, welcome to the course!
- Next steps:
  1. **Let's hear any questions you have**
  2. Once everything is clear, let's start the course! 🧙
- Don't be afraid to contact me during the course regarding any questions you have